PROJECT-BASED LEARNING

PHASE-II EVALUATION

Multi-Drone Search & Rescue for Dynamic Disaster Response Using Graph-Based Path Planning and Priority Task Allocation

Team ID: DSCPP-III-2026-T290

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T290
Semester: 3rd
Section: DS2, DS1, J, CYBER
Domain: Autonomous Systems UAV and Intelligent Disaster Response

Team Members

1. JAYNA KOTHARI - 2510380106
2. HRIMANDEEP SINGH - 2510380210
3. AARAV FANGALIA - 2510370262
4. RAGHAV SINGHAL - 2510340011

Mentor

(Prof).Dr.Amit Kumar

Interactions completed: 05

Phase-I Recap & Feedback Incorporated

Phase-I Status

The project proposes a graph-based Multi-Drone Search and Rescue system for disaster response.
It uses graph representation for connecting locations, priority-based mission scheduling, shortest-path planning using Dijkstra's algorithm, and UAV battery and failure constraints. This system aims to assign suitable drones to missions and support dynamic reassignment when conditions change.

Feedback Received

- Improve mission allocation using priority levels.
- Handle dynamic situations such as UAV failure and blocked routes.
- Consider UAV battery levels and mission feasibility before assignment.

Changes Incorporated

- Introduced priority-based mission scheduling to process urgent missions first.
- Added battery-feasibility checks and graph-based route planning using Dijkstra's algorithm and BFS for traversal/reachability.
- Designed dynamic reassignment logic to handle unavailable drones and blocked routes.

Problem Statement & Finalized Objectives

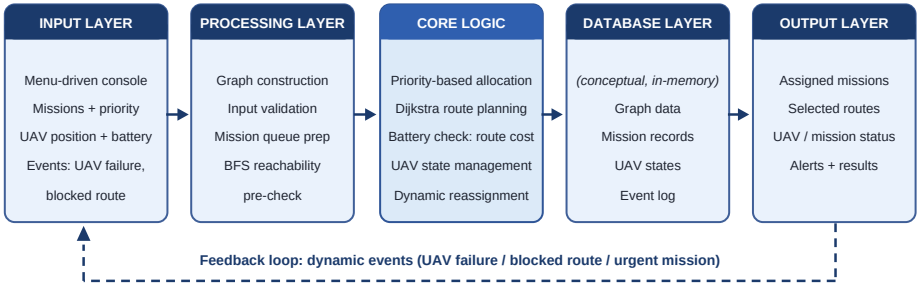
Problem Statement

Disaster situations require rapid coordination of multiple UAVs to handle rescue missions efficiently. Manual mission allocation may cause delays, inefficient routing, and difficulty managing drone failures or blocked paths. This project aims to develop a graph-based multi-drone search and rescue simulation using priority-based task allocation, shortest-path planning, battery feasibility checks, and dynamic mission reassignment.

Finalized Objectives

1. Represent disaster locations and routes using a weighted graph and adjacency list.
2. Allocate missions according to priority using a priority queue.
3. Apply BFS for graph traversal/reachability and Dijkstra's algorithm for minimum-cost route planning.
4. Simulate UAV battery constraints, drone failures, blocked routes, and dynamic mission reassignment.

Updated System Architecture / Workflow



Major Components

- Graph & PathPlanning – Graph representation, BFS and Dijkstra's algorithm.
- Mission Allocation Module – Priority queue-based mission scheduling.
- UAV Coordination Module – Battery feasibility, drone status and dynamic mission reassignment.

Changes Since Phase-I

- Added priority-based mission scheduling.
- Incorporated battery-feasibility and UAV failure handling.
- Improved route planning and dynamic reassignment for blocked routes.

Module Design & Functional Requirements

Major Modules

1. **Graph & Path Planning**: Represents locations and routes using an adjacency list; uses BFS and Dijkstra's algorithm.
2. **Mission Management**: Stores mission details and schedules tasks using a priority queue.
3. **UAV Management**: Maintains drone position, battery level, availability and operational status.
4. **Coordinator & Event Handling**: Assigns eligible drones and manages reassignment during failures or blocked routes.

Functional Requirements

- Accept mission details, priorities and destination locations.
- Find reachable locations and minimum-cost routes.
- Assign the highest-priority pending mission to an eligible UAV.
- Handle battery constraints, UAV failures and blocked routes.

Module Dependency

The Mission Management module sends prioritized tasks to the Coordinator. The Graph module provides route information, while UAV Management checks drone availability and battery feasibility. The Coordinator combines these inputs to assign missions and manage reassignment when dynamic events occur.

Technology Stack & Development Methodology

Technology Stack

Programming: C / C++
Database: Not Required (Simulation Data)
Framework: Standard C++ Libraries (STL)
Development Tool: Visual Studio Code
Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis – Identify mission priorities, UAV battery constraints, route planning and dynamic disaster scenarios.
2. System Design – Design the weighted graph, adjacency list, priority queue and C++ classes for UAV, Mission, Graph and Coordinator.
3. Module Development – Implement BFS, Dijkstra's algorithm, priority-based scheduling and UAV eligibility checks.
4. System Integration – Connect graph planning, mission allocation and UAV management through the Coordinator module.
5. Testing & Validation – Test mission priorities, route selection, battery feasibility, blocked routes and UAV failure scenarios.

Implementation of PBL Course Concepts

Course	Concepts actually implemented	Project module
Data Structures in C	Graph (adjacency list), BFS, Dijkstra's shortest path, priority queue (heap), queue, 2D matrix for blocked routes, arrays/vectors	Module X: Route planning and graph
OOPs with C++	Classes (UAV, Mission, Graph, Coordinator), encapsulation, constructors, operator overloading (comparator), STL (vector, map, priority_queue), smart pointers (unique_ptr), lambdas	Module Y: UAV and mission management
Operating Systems	Cross-platform development and execution on Windows and macOS (portable standard C++, no OS-specific APIs), console-based I/O	Whole project (Modules X and Y)
DBMS	Not required for this project (data is handled in memory)	Not required

Development & Implementation Progress

Module	Status	Work completed
Module 1: Graph and route planning	Completed	Adjacency-list graph, BFS reachability check, Dijkstra shortest path, route blocking
Module 2: UAV and mission management	Completed	UAV and Mission classes, priority queue, best-UAV selection by route cost and battery
Module 3: Dynamic event handling	Completed	Low battery, UAV failure, route blocking, urgent missions, automatic reassignment, mission completion
Module 4: Code finalization and bug fixing	Pending	Core code written and tested (8 of 8 test cases passed); remaining: menu input-error fix and final cleanup
Module 5: Data storage and GUI	Pending	Planned: save UAV and mission data to a database, graphical interface (not started)

Overall project completion: **60%** (3 of 5 modules)

Working Prototype / Demonstration

Implemented features

- Automatic mission allocation by priority and shortest route
- Dynamic events: low battery, UAV failure, blocked routes
- Urgent mission insertion (priority 5) with instant assignment
- Shortest route finder using BFS and Dijkstra

Prototype / output

```
[ALERT] Mission M1 affected by UAV failure.
[SYSTEM] Mission returned to priority queue.
[EVENT] UAV-F has FAILED.

[AUTO ALLOCATION]
Mission: M1 | Priority: 5 | Type: Medical
Selected UAV: UAV-L
[SUCCESS] Mission M1 assigned to UAV-L.

Dijkstra Shortest Path: 1 -> 3 -> 4 -> 5 -> 6
Total Cost: 7
```

Testing, Results & Validation

Test / module	Expected result	Observed result	Status
1. Initial allocation (3 missions)	Each mission gets a UAV	M1 to UAV-F, M2 to UAV-D, M3 to UAV-E	Pass
2. Shortest route 1 to 6	Lowest-cost path	1-3-4-5-6, cost 7	Pass
3. Block route 4-5, route 1 to 5	Path avoids blocked edge	Cost 6 changed to 8 via 1-3-5	Pass
4. Block 4-6 and 5-6, route 1 to 6	Node 6 unreachable	BFS: destination NOT reachable	Pass
5. UAV-F fails mid-mission	Mission requeued and reassigned	M1 reassigned to UAV-L	Pass
6. All 12 UAVs fail	Mission stays pending	M1 Pending, "No suitable UAV available"	Pass
7. Low battery set to 10%	UAV marked low battery	UAV-F shows LOW BATTERY	Pass
8. Duplicate mission ID	Second entry rejected	"Mission ID already exists"	Pass

Results / validation

8 of 8 test cases passed on a 6-node map with 12 UAVs. Event handling and re-routing were checked by running the compiled program.

GitHub Repository & Development Activity

Repository Information

- GitHubLink: RepositoryURL
- README.md: Updated Project overview, objectives, workflow and references documented
- Contributors: All Members Added
- Repository Status: Updated
- Branches: main (default branch)

Development Activity

- Regular commits maintained
- Meaningful commit messages
- Individual contribution visible
- Latest implementation uploaded
- Documentation maintained

Individual Contribution

Member	Role	Technical contribution	GitHub commits
Aarav Fangalia	Problem model and coordinator	Finalised problem statement, formal model and UAV eligibility/assignment rule; coordinator allocation loop; GitHub repository set-up and structure	4
Jayna Kothari	Graph and routing	Weighted-graph adjacency list, BFS reachability, heap-based Dijkstra with predecessor tracking (verified by hand trace)	[XX]
Raghav Singhal	UAV, missions and events	Priority-queue mission handler with sequence numbers; UAV and Mission classes; dynamic event handler for low battery, failure, blocked route and urgent mission	[XX]
Hrimandeep Singh	Testing, results and documentation	Five analytic test scenarios and verification table; graph, Dijkstra tree, cost and success-rate figures; dashboard concept; research paper; GitHub commit history	7

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Technical challenge: Finding feasible routes while considering weighted edges and UAV battery limits.
- Integration challenge: Connecting graph algorithms, mission scheduling and UAV management.
- Dynamic events: Handling drone failure, blocked routes and urgent missions.

Phase-III Roadmap

1. Complete and verify remaining modules.
2. Integrate mission scheduling, path planning and UAV management.
3. Test battery constraints, blocked routes and drone failure scenarios.
4. Validate results and optimize mission reassignment.
5. Finalize documentation, demonstration and project report.

Solutions / Learning

➤ Solution adopted: Modular C++ classes and separate logic for routing, scheduling and coordination.

➤ Technical learning: Improved understanding of graphs, BFS, Dijkstra's algorithm, priority queues and object-oriented programming.

Expected Outcomes & References

Expected Final Outcomes

➤ Working Prototype: C++-based multi-drone rescue simulation.
➤ Technical Results: Route cost, mission priority and UAV eligibility validation.
➤ Practical Application: Coordinated mission allocation for disaster-response scenarios.
➤ Scalability Potential: Extendable to larger graphs and additional UAVs.
➤ Future Research: Integration with real drone hardware and advanced rescue systems.

References

1. E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.
2. T. H. Cormen et al., Introduction to Algorithms, 4th ed., MIT Press, 2022.
3. Bjarne Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.
4. cppreference.com, “C++ Standard Library: Containers and priority_queue.”
5. Project Repository: https://github.com/hrimandeepsingh/multi-drone-search-rescue-PBL